

Assignment II: Mini-RISC CPU Simulator with Explicit Control Unit

Department of Computer Science and Engineering

Objective

In Assignment I you implemented an ISA-level simulator. In this assignment you will build a **micro-architectural simulator** by introducing an explicit **datapath**, a **Control Unit**, and **control signals**. You will execute every instruction using a **multi-cycle FSM** (FETCH, DECODE, EXEC, WRITEBACK).

You will learn the difference between **ISA** and **micro-architecture** (**datapath** + **control**).

CPU Architecture Overview

The CPU is split into two parts:

- **Datapath:** PC, IR, ACC, Register File (R0–R3), ALU, Flags, Instruction Memory
- **Control Unit:** Generates control signals based on (state, opcode)

Registers and Storage

- R0, R1, R2, R3 (8-bit each)
- ACC (8-bit), PC (8-bit), IR (8-bit)
- Flags: Z (Zero), C (Carry)
- Instruction Memory: $256 \times 8 - bit$

ALU

Supported operations: ADD, SUB, MUL, DIV, POW, CMP.

Control Signals

- PC_{write} ,
- IR_{write}

- ACC_{write}
- REG_{write}
- $MEM_{read},$
- $ALU_{op},$

Multi-cycle FSM

States: **S0: FETCH**, **S1: DECODE**, **S2: EXEC**, **S3: WRITEBACK**.

- **FETCH:** $IR \leftarrow MEM[PC], PC \leftarrow PC + 1$
- **DECODE:** Decode opcode/operand
- **EXEC:** ALU executes or target prepared
- **WRITEBACK:** Write result to ACC/Reg or update PC

Instruction Set (same as Assignment I)

Opcode	Mnemonic	Operation
0x0	NOP	No operation
0x1	LOAD Rn	$ACC \leftarrow R_n$
0x2	STORE Rn	$R_n \leftarrow ACC$
0x3	MOVI k	$ACC \leftarrow k$
0x4	ADD Rn	$ACC \leftarrow ACC + R_n$
0x5	SUB Rn	$ACC \leftarrow ACC - R_n$
0x6	MUL Rn	$ACC \leftarrow ACC \times R_n$
0x7	DIV Rn	$ACC \leftarrow ACC \div R_n$
0x8	POW Rn	$ACC \leftarrow ACC^{R_n}$
0x9	CMP Rn	Set flags based on $(ACC - R_n)$
0xA	JMP addr	$PC \leftarrow \text{addr}$
0xB	JZ addr	If $Z = 1$, $PC \leftarrow \text{addr}$
0xC	JC addr	If $C = 1$, $PC \leftarrow \text{addr}$
0xF	HALT	Stop execution

New Rule

You must not execute instructions in a single switch-case. All effects must occur via **control signals** + **datapath** across FSM states.

C Program Skeleton

```

1  /* =====
2  Mini-RISC CPU Simulator      Assignment II (With Control Unit)
3  ===== */
4
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <string.h>
8
9  #define MEM_SIZE 256
10
11 // ===== CPU STATE =====
12
13 typedef struct {
14     unsigned char R[4];
15     unsigned char ACC;
16     unsigned char PC;
17     unsigned char IR;
18     unsigned char Z;
19     unsigned char C;
20 } CPU;
21
22 // ===== CONTROL SIGNALS =====
23
24 typedef struct {
25     int PC_write;
26     int IR_write;
27     int ACC_write;
28     int REG_write;
29     int MEM_read;
30     int MEM_write;
31     int ALU_op;
32     int ALU_src;
33     int PC_src;
34 } ControlSignals;
35
36 // ===== FSM STATES =====
37
38 typedef enum {
39     S_FETCH,
40     S_DECODE,
41     S_EXEC,
42     S_WB,
43     S_HALT
44 } CPUState;
45
46 unsigned char MEMORY[MEM_SIZE];
47
48 // ===== PROTOTYPES =====
49

```

```

50 void reset_cpu(CPU *cpu);
51 void load_program(const char *fname);
52 void generate_control_signals(CPU *cpu, CPUState state,
    ControlSignals *ctrl);
53 void apply_control_signals(CPU *cpu, ControlSignals *ctrl);
54 unsigned char alu_execute(unsigned char a, unsigned char b, int
    alu_op, CPU *cpu);
55
56 // ===== MAIN =====
57
58 int main() {
59     CPU cpu;
60     ControlSignals ctrl;
61     CPUState state = S_FETCH;
62
63     reset_cpu(&cpu);
64     load_program("program.hex");
65
66     int running = 1;
67
68     while (running) {
69         memset(&ctrl, 0, sizeof(ControlSignals));
70
71         generate_control_signals(&cpu, state, &ctrl);
72         apply_control_signals(&cpu, &ctrl);
73
74         switch (state) {
75             case S_FETCH: state = S_DECODE; break;
76             case S_DECODE: state = S_EXEC; break;
77             case S_EXEC: state = S_WB; break;
78             case S_WB:
79                 if (((cpu.IR >> 4) & 0xF) == 0xF) state = S_HALT;
80                 else state = S_FETCH;
81                 break;
82             case S_HALT: running = 0; break;
83         }
84     }
85
86     printf("Program finished.\n");
87     return 0;
88 }
89
90 // ===== STUDENTS IMPLEMENT BELOW =====
91
92 void generate_control_signals(CPU *cpu, CPUState state,
    ControlSignals *ctrl) {
93     // TODO: Based on state and opcode, set control signals
94 }
95
96 void apply_control_signals(CPU *cpu, ControlSignals *ctrl) {

```

```
98 // TODO: Perform datapath actions based on control signals
99 }
100
101 unsigned char alu_execute(unsigned char a, unsigned char b, int
    alu_op, CPU *cpu) {
102 // TODO: Implement ALU ops and set flags
103 return 0;
104 }
105
106 void reset_cpu(CPU *cpu) {
107 memset(cpu, 0, sizeof(CPU));
108 }
109
110 void load_program(const char *fname) {
111 FILE *fp = fopen(fname, "r");
112 if (!fp) { printf("Cannot open program file!\n"); exit(1); }
113 int addr = 0;
114 unsigned int x;
115 while (fscanf(fp, "%x", &x) != EOF) {
116 MEMORY[addr++] = (unsigned char)x;
117 }
118 fclose(fp);
119 }
```